

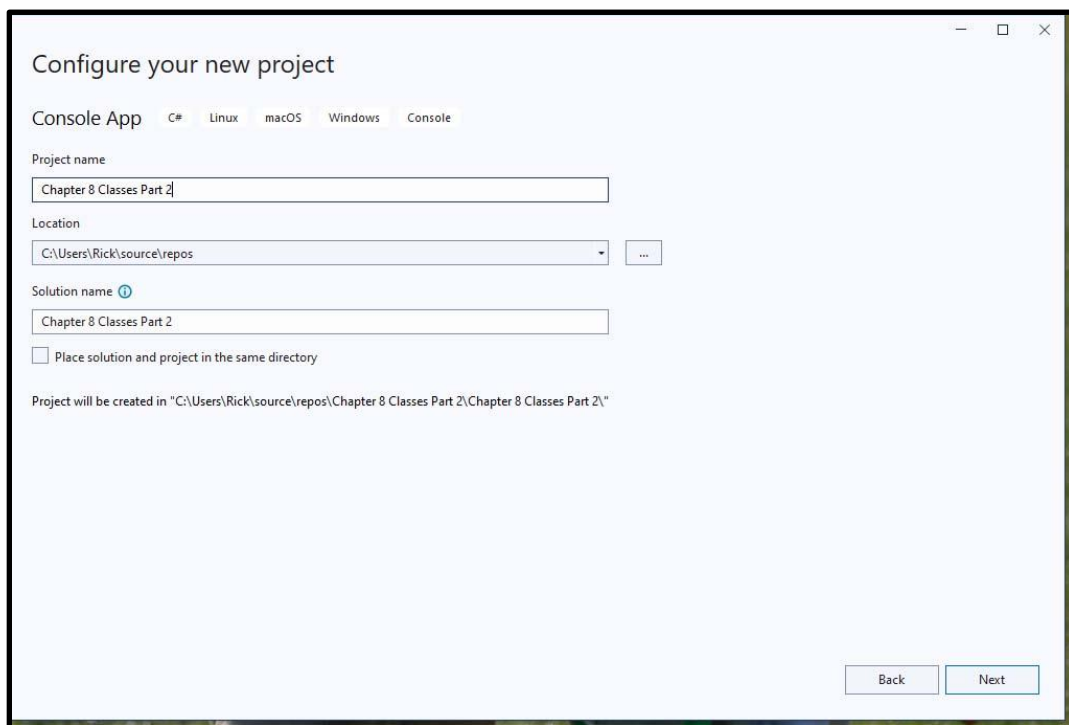
Chapter 8: Classes Part 2

In this chapter we continue our look at classes. In particular we are going to look at required versus optional properties and how to specify those properties in the class constructor. We are also going to take a look at class inheritance, i.e. creating a more complex class from a simpler one.

Let's jump to the inside and start looking at some examples.

We begin of course with the creation of a new console project named Chapter 8 Classes Part 2. (Figure 8.1)

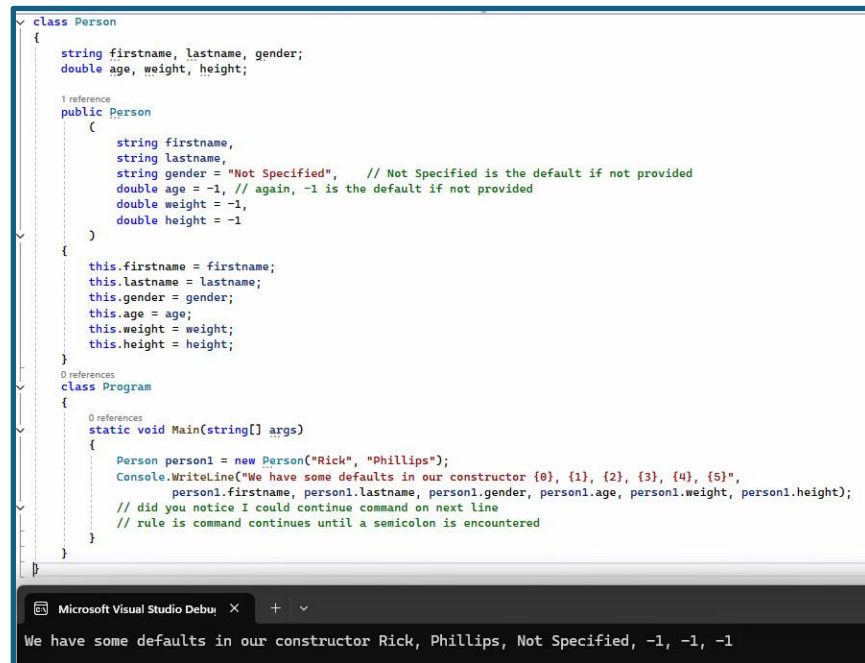
Figure 8.1: Project Creation



We start with the example shown here. (Figure 8.2) A couple of things in this example warrant our attention:

- First, please note how we have specified a default value for the gender, age, weight and height properties. Please also check out the comment. It is pretty common practice to use the string “Not Specified” as the default for string parameters and -1 for numeric parameters.
- Next, please note the use of the keyword `this`. The keyword `this` refers to the current instance of a class. I didn’t use this syntax often, (yes, all puns intended) but I know a lot of people do so it is important to know it.
- Next, note the use of the same private variable names as the passed-in parameters. Nice way to make your code more self-documenting. Again unfortunately, this is yet another standard not always adhered to by many programmers. As someone who doesn’t type as well as I used to, I am afraid I too am guilty of typically using shortened versions of the parameters as the internal variable names.
- Finally, please note how I was able to continue my `Console.WriteLine` command on the next line. The rule here is real simple. The command does not end until a semicolon is encountered.

Figure 8.2: Defaults and this Keyword



```
class Person
{
    string firstname, lastname, gender;
    double age, weight, height;

    1 reference
    public Person
    (
        string firstname,
        string lastname,
        string gender = "Not Specified", // Not Specified is the default if not provided
        double age = -1, // again, -1 is the default if not provided
        double weight = -1,
        double height = -1
    )
    {
        this.firstname = firstname;
        this.lastname = lastname;
        this.gender = gender;
        this.age = age;
        this.weight = weight;
        this.height = height;
    }
}

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Person person1 = new Person("Rick", "Phillips");
        Console.WriteLine("We have some defaults in our constructor {0}, {1}, {2}, {3}, {4}, {5}",
            person1.firstname, person1.lastname, person1.gender, person1.age, person1.weight, person1.height);
        // did you notice I could continue command on next line
        // rule is command continues until a semicolon is encountered
    }
}
```

Microsoft Visual Studio Debug Console

We have some defaults in our constructor Rick, Phillips, Not Specified, -1, -1, -1

In our next example we take a look at a way to check within the constructor itself whether all the required parameters are present. (Figure 8.3) Much more elegant than the previous solution which just throws a compilation error. By the way, the Console.WriteLine text is correct. We will learn how to throw an error back to the calling routine in the next chapter so stay tuned.

Figure 8.3: Checking for Required Fields in Constructor

```
class Person
{
    string firstname, lastname, gender;
    double age, weight, height;

    2 references
    public Person
    (
        string firstname = "Not Specified",
        string lastname = "Not Specified",
        string gender = "Not Specified", // Not Specified is the default if not provided
        double age = -1, // again, -1 is the default if not provided
        double weight = -1,
        double height = -1
    )
    {
        if (firstname == "Not Specified" || lastname == "Not Specified")
        {
            Console.WriteLine("Throw an error back to the calling routine indicating missing parms");
            Console.WriteLine("We will learn how to do this in the next lesson");
        }

        this.firstname = firstname;
        this.lastname = lastname;
        this.gender = gender;
        this.age = age;
        this.weight = weight;
        this.height = height;
    }
}

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Person person1 = new Person("Rick", "Phillips");
        Person person2 = new Person(firstname: "Rick"); // missing parm should invoke error handler
        Console.WriteLine("We have some defaults in our constructor {0}, {1}, {2}, {3}, {4}, {5}",
            person1.firstname, person1.lastname, person1.gender, person1.age, person1.weight, person1.height);
        // did you notice I could continue command on next line
        // rule is command continues until a semicolon is encountered
    }
}
```

Microsoft Visual Studio Debug Console

```
Throw an error back to the calling routine indicating missing parms
We will learn how to do this in the next lesson
We have some defaults in our constructor Rick, Phillips, Not Specified, -1, -1, -1
```

In our next example we start to explore the power of class inheritance with the Father, Mother and Child classes all inheriting from the Family class. In this example the Family class is known as the base class while the Father, Mother and Child classes are called derived classes. A derived class may only inherit from a single base class, but inheritance may be to any depth. I wanted to show you a slightly different arrangement of the Program, Main and class definitions with this example so I created a second project named Chapter 8 part 2b. As such, you will find two code files associated with this lesson.

We begin with the code shown here defining our various classes. (Figure 8.4) Note the syntax for inheritance. The Father class inherits from the Family class, so the syntax reads `class Father : Family`. Please also note the abbreviated syntax I used to create the Family class constructor. This get/set syntax is very common, and you will see it often. Finally, please note how I have overloaded the constructors for our Father, Mother and Child classes. Only the one which takes a Family class input parameter is going to give us inheritance. The base constructor would be appropriate if we were just creating the Father, Mother or Child class without the associated inherited base class (Family).

Figure 8.4: Inheritance Syntax

```
2 references
class Father : Family
{
    private string fn, mi;

    // base constructor
    0 references
    public Father() { }

    // overloaded constructor
    0 references
    public Father (Family incomingFamily)
    {
        base.LastName = incomingFamily.LastName;
    }
}

2 references
class Mother : Family
{
    private string fn, mi;

    // base constructor
    0 references
    public Mother() { }

    // overloaded constructor
    0 references
    public Mother(Family incomingFamily)
    {
        base.LastName = incomingFamily.LastName;
    }
}

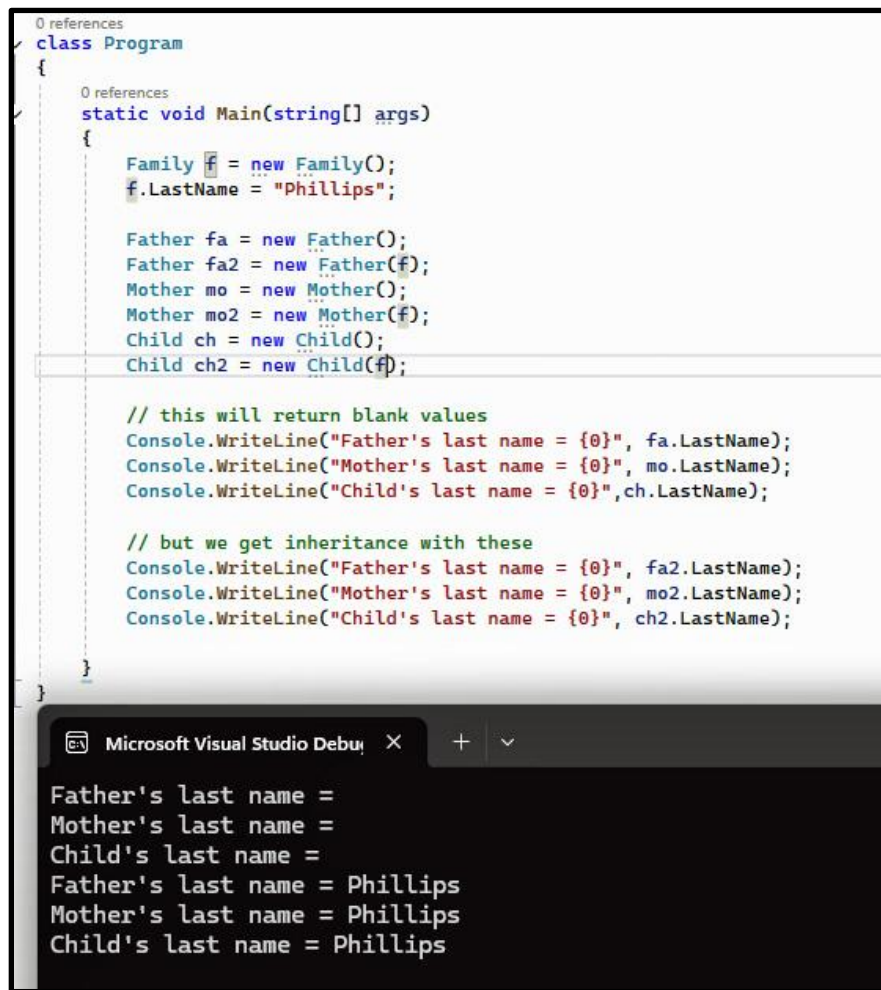
2 references
class Child : Family
{
    private string fn, mi;

    // base constructor
    0 references
    public Child() { }

    // overloaded constructor
    0 references
    public Child(Family incomingFamily)
    {
        base.LastName = incomingFamily.LastName;
    }
}
```

This image shows my associated calling routine and its output. (Figure 8.5)

Figure 8.5: Inheritance Calling Routine and Output



```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Family f = new Family();
        f.LastName = "Phillips";

        Father fa = new Father();
        Father fa2 = new Father(f);
        Mother mo = new Mother();
        Mother mo2 = new Mother(f);
        Child ch = new Child();
        Child ch2 = new Child(f);

        // this will return blank values
        Console.WriteLine("Father's last name = {0}", fa.LastName);
        Console.WriteLine("Mother's last name = {0}", mo.LastName);
        Console.WriteLine("Child's last name = {0}", ch.LastName);

        // but we get inheritance with these
        Console.WriteLine("Father's last name = {0}", fa2.LastName);
        Console.WriteLine("Mother's last name = {0}", mo2.LastName);
        Console.WriteLine("Child's last name = {0}", ch2.LastName);
    }
}
```

Microsoft Visual Studio Debug Console

```
Father's last name =
Mother's last name =
Child's last name =
Father's last name = Phillips
Mother's last name = Phillips
Child's last name = Phillips
```

In this next series of images (Figure 8.6a through c) I have added two parameterized constructors to our Father, Mother and Child classes. In addition, I have added a method which returns an individual's full name via the concatenation of the fn, mi and LastName properties. Just as an aside, have you noticed how our code is getting much longer. By the time we add in the Main routine this one will have taken 4 screen shots.

Figure 8.6a: Two-Parameterized Constructor

```
class Father : Family
{
    private string fn, mi;

    // base constructor
    2 references
    public Father(string first_name, string middle_initial)
    {
        fn = first_name;
        mi = middle_initial;
    }

    // overloaded constructor
    2 references
    public Father (string first_name, string middle_initial, Family incomingFamily)
    {
        fn = first_name;
        mi = middle_initial;
        base.LastName = incomingFamily.LastName;
    }

    0 references
    public string concatNames()
    {
        if (mi.Length > 0) return fn + " " + mi + " " + LastName;
        else return fn + " " + LastName;
    }
}
```

Figure 8.6b: Two-Parameterized Constructor

```
6 references
class Mother : Family
{
    private string fn, mi;

    // base constructor
    2 references
    public Mother(string first_name, string middle_initial)
    {
        fn =first_name;
        mi = middle_initial;
    }

    // overloaded constructor
    2 references
    public Mother(string first_name, string middle_initial, Family incomingFamily)
    {
        fn = first_name;
        mi = middle_initial;
        base.LastName = incomingFamily.LastName;
    }

    0 references
    public string concatNames()
    {
        if (mi.Length > 0) return fn + " " + mi + " " + LastName;
        else return fn + " " + LastName;
    }
}
```

Figure 8.6c: Two-Parameterized Constructor

```
6 references
class Child : Family
{
    private string fn, mi;

    // base constructor
    2 references
    public Child(string first_name, string middle_initial)
    {
        fn = first_name;
        mi = middle_initial;
    }

    // overloaded constructor
    2 references
    public Child(string first_name, string middle_initial, Family incomingFamily)
    {
        fn = first_name;
        mi = middle_initial;
        base.LastName = incomingFamily.LastName;
    }

    0 references
    public string concatNames()
    {
        if (mi.Length > 0) return fn + " " + mi + " " + LastName;
        else return fn + " " + LastName;
    }
}
```

Now before we discuss my changes to the Main routine, I want you to think about what we have here even with this very simple example. I now have a situation where I can instantiate the Family class alone, any of the Father, Mother and Child classes alone or finally the Father, Mother and Child classes with the inherited Family class.

I think we can all agree that the output shown here substantiates my last statement. (Figure 8.7) This is the real power of object-oriented programming, i.e. classes, when combined with inheritance it results in real flexibility. Great stuff.

Figure 8.7: Updated Main and Output

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Family f = new Family();
        f.LastName = "Phillips";

        Father fa = new Father("Rick", "L");
        fa.LastName = "Different";
        Father fa2 = new Father("Ben", "D", f);

        Mother mo = new Mother("Margret", "");
        Mother mo2 = new Mother("Margret", "", f);

        Child ch = new Child("Daniel", "A");
        Child ch2 = new Child("Daniel", "A", f);

        // this will return blank values
        Console.WriteLine("Father's last name = {0}", fa.LastName);
        Console.WriteLine("Mother's last name = {0}", mo.LastName);
        Console.WriteLine("Child's last name = {0}", ch.LastName);

        // but we get inheritance with these
        Console.WriteLine("Father's last name = {0}", fa2.LastName);
        Console.WriteLine("Mother's last name = {0}", mo2.LastName);
        Console.WriteLine("Child's last name = {0}", ch2.LastName);

        // call our new method on fa and fa2
        Console.WriteLine("fa's full name is {0}", fa.concatNames());
        Console.WriteLine("fa2's full name is {0}", fa2.concatNames());
    }
}
```

Microsoft Visual Studio Debug Console

```
Father's last name = Different
Mother's last name = 
Child's last name = 
Father's last name = Phillips
Mother's last name = Phillips
Child's last name = Phillips
fa's full name is Rick L Different
fa2's full name is Ben D Phillips
```

Okay, let's jump to the outside and do a quick review.

So, what did we learn?

- We started with parameterized constructors and optional versus required parameters. Super useful stuff we will use all the time going forward. It was always my standard way of making a constructor because it makes your code more self-documenting.
- We next learned how to check for required parameters in a constructor. In our next lesson we will learn how to send that error message back to the calling routine.

- Our next example was that simple inheritance code where we had a Family base class and Father, Mother and Child derived classes. It was okay, but the constructors in the first version were too simple.
- We beefed those constructors up in the last example where we put a parameterized constructor on the base class and overloaded the constructors on the derived classes.

All in all, a successful look at classes and inheritance. In our next lesson we will take a look at namespaces and error handlers.